REQUERIMIENTOS Y ORGANIZACIÓN DEL CÓDIGO
MINI PROYECTO DE EVALUACIÓN – RESULTADO DE APRENDIZAJE 1 (RA1)

Aplicaciones Web con JavaScript | Trabajo en equipos de dos (2) integrantes

Valor del mini proyecto: 5 puntos

El presente documento establece los **requisitos mínimos obligatorios** de entrega, así como las normas de organización y calidad del código fuente que deberán cumplir los proyectos desarrollados por los equipos, tanto en la capa de **frontend** como en la de **backend**. Estas disposiciones aplican por igual a los proyectos de videojuegos y a las aplicaciones comerciales, dado que todos han sido orientados a la web mediante el lenguaje de programación JavaScript. **El incumplimiento de estas normas afectará directamente la calificación obtenida.**

1. Requisitos generales de entrega

- Repositorio **Git** (GitHub, GitLab o Bitbucket) con historial de commits de **ambos integrantes**, como evidencia del trabajo colaborativo.
- Archivo **README.md** que contenga: nombre del proyecto, integrantes del equipo, descripción funcional, requisitos previos, pasos de instalación y ejecución, y capturas de pantalla de la aplicación en funcionamiento.
- Archivo **.gitignore** que excluya `node_modules/`, `.env`, `dist/` y archivos temporales.
- Variables de configuración almacenadas en un archivo **.env**, acompañado de un **.env.example** versionado. **Queda terminantemente prohibido incluir credenciales dentro del código fuente.**
- Convención de nomenclatura uniforme: carpetas y archivos en *kebab-case*; variables y funciones en *camelCase*; clases en *PascalCase*; constantes en *UPPER_SNAKE_CASE*.
- Un solo idioma para todo el código fuente y sus comentarios. Se recomienda inglés para el código y español para la documentación.

2. Organización del frontend

Estructura mínima obligatoria de carpetas:

```
frontend/
|-- index.html
|-- src/
|   |-- css/          estilos separados (sin <style> ni style="")
|   |-- js/
|   |   |-- main.js   punto de entrada, unicamente orquesta
|   |   |-- modules/  logica dividida por responsabilidad
|   |   |-- services/ comunicacion con el backend (fetch / axios)
|   |   |-- utils/    funciones auxiliares reutilizables
|   |-- assets/       imagenes, sonidos, fuentes
```

2.1 Normas de obligatorio cumplimiento

- Uso de **módulos ES6** (`import` / `export`). Un archivo por responsabilidad. **Queda prohibido entregar un único archivo con todo el código.**
- **No se permite** el uso de `onclick="..."` u otros manejadores de eventos dentro del HTML. Los eventos se registran desde JavaScript mediante `addEventListener`.
- **Prohibidas las variables globales sueltas.** Toda información debe encapsularse en módulos, clases u objetos.
- Toda comunicación con el backend se centraliza en la carpeta **services/**, con una función por cada punto de acceso. No se admiten llamadas `fetch` dispersas en las vistas.
- Diseño **responsivo**, verificable en dispositivos móviles y de escritorio.

2.2 Requisitos adicionales para videojuegos

- **engine/** o **core/**: bucle principal del juego, control del tiempo y renderizado.
- **entities/**: una clase por entidad (Player, Enemy, Bullet, entre otras).
- **scenes/** o **states/**: menú, juego, pausa y fin de partida.
- **input/**: manejo de teclado, ratón y controles táctiles, aislado del resto de la lógica.
- La lógica del juego **no debe depender directamente del DOM ni del canvas**; el renderizado se mantiene separado.

2.3 Requisitos adicionales para aplicaciones comerciales

- **views/** o **components/**: cada pantalla o componente en su propio archivo.
- **Validación de formularios** tanto en el cliente como en el servidor.
- Formato de moneda, fechas e **ITBIS** centralizado en funciones de **utils/**, nunca replicado en distintas pantallas.

3. Organización del backend

Estructura por capas (Node.js con Express o tecnología equivalente):

```
backend/
|-- server.js           unicamente levanta el servidor
|-- src/
|   |-- config/         conexion a BD y variables de entorno
|   |-- routes/         define rutas, sin logica de negocio
|   |-- controllers/    recibe la peticion, responde y delega
|   |-- services/       reglas de negocio
|   |-- repositories/   acceso a datos y consultas
|   |-- models/         entidades o esquemas
|   |-- middlewares/    autenticacion, validacion y errores
|   |-- utils/
|-- package.json
```

3.1 Normas de obligatorio cumplimiento

- **Separación estricta de responsabilidades:** la ruta no consulta la base de datos, el controlador no construye sentencias SQL y el servicio no manipula los objetos `req` / `res`.
- API **REST** con rutas en plural y verbos HTTP correctos: `GET /api/products`, `POST /api/products`, `PUT /api/products/:id`, `DELETE /api/products/:id`.
- Uso apropiado de los **códigos de estado HTTP**: 200, 201, 400, 401, 404 y 500.

- Respuestas en formato JSON uniforme, por ejemplo: `{ success, data, message }`.
- **Middleware global** para el manejo de errores. No se admite devolver al cliente el error interno sin procesar.
- **Validación de los datos de entrada en el servidor** (express-validator, Zod o validación manual debidamente documentada).
- **Consultas parametrizadas** para prevenir la inyección SQL.
- Configuración explícita de **CORS**.

En el caso de los videojuegos que no contemplen un backend propio, se exigirá como mínimo un servicio de apoyo: registro de jugadores, guardado de partidas o tabla de puntuaciones (*leaderboard*) persistida en base de datos.

4. Normas contra el código espagueti

Los siguientes criterios son **verificables de forma objetiva** durante la revisión del código fuente:

- Ninguna función podrá exceder las **40 líneas**; de ser necesario, deberá dividirse.
- Ningún archivo podrá exceder las **300 líneas**.
- Máximo **tres (3) niveles de anidación** de estructuras de control.
- **Sin código duplicado:** todo bloque que se repita dos o más veces debe extraerse a una función.
- **Sin código comentado** ni instrucciones `console.log` de depuración en la entrega final.
- **Sin números mágicos:** deberán emplearse constantes con nombre descriptivo.
- Cada función realiza **una sola tarea** y su nombre lo refleja (verbo más sustantivo: `calculateTotal`, `renderPlayer`).
- Indentación consistente. Se recomienda configurar **Prettier** y **ESLint** en el proyecto.

5. Entregables el día de la presentación

1. Enlace al **repositorio** del proyecto.
2. Aplicación **desplegada y funcional** (Netlify o Vercel para el frontend; Render o Railway para el backend), o demostración local sin errores.
3. **Documento técnico** breve de tres (3) a cinco (5) páginas: descripción del proyecto, diagrama de la arquitectura de carpetas, diagrama de la base de datos y descripción de los puntos de acceso de la API.
4. **Video** de cinco (5) a ocho (8) minutos en el que **ambos integrantes** expliquen el funcionamiento y la estructura del código.
5. **Defensa oral:** cada integrante deberá estar en capacidad de explicar cualquier parte del código, no únicamente la sección que desarrolló.

6. Distribución de la calificación (5 puntos)

Criterio de evaluación	Puntos
Funcionalidad completa y sin errores críticos	1.25
Organización y estructura del código (carpetas, capas, modularidad)	1.25
Calidad del código (nomenclatura, funciones breves, sin duplicación)	0.75
Backend: API, validación y manejo de errores	0.75
Documentación (README y documento técnico)	0.50
Presentación y defensa individual	0.50
TOTAL	**5.00**

7. Disposiciones finales

- El equipo deberá entregar, junto con el proyecto, una **autoevaluación** en la que marque el cumplimiento de cada uno de los requisitos contenidos en este documento.
- **No se aceptarán entregas fuera de la fecha establecida** ni proyectos que no puedan ser ejecutados durante la presentación.
- La calificación es **individual**: el integrante que no demuestre dominio del código presentado recibirá una puntuación diferenciada.

Documento de uso académico. Elaborado para la evaluación del Resultado de Aprendizaje 1 (RA1).